

SESIÓN 5

Contenerización con Docker y Docker Compose

Informe de ejecución de los tres laboratorios

Estudiante	Albert Omar Araya Cory
Repositorio	github.com/albrtaraya/fullstack-lab-1-1ci-cd
Pull Request	#8 — integrado a main (df837e1)
Servicios	postgres · backend · frontend
Docker	Engine 27.5.1 · Compose v2.32.4
Modalidad	Individual
Fecha	14 de septiembre de 2026

Insignias

 **Maestro de Compose (15 pts)**

 **Arquitecto del Barco Completo (20 pts) — pendiente de la prueba con un compañero**

Resumen de ejecución

La Sesión 5 abre la Unidad 3: la aplicación completa — frontend, backend y base de datos — pasa a levantarse con un único comando, sin instalar Node ni PostgreSQL en la máquina.

Laboratorio	Nombre	Tiempo	Estado
1	Dockerfile del backend	35 min	Cumplido
2	Dockerfile multi-etapa del frontend	30 min	Cumplido
3	El Puerto de Embarque: docker-compose.yml	45 min	Cumplido

Prerrequisitos verificados

```
$ docker --version
Docker version 27.5.1, build 9f9e405

$ docker compose version
Docker Compose version v2.32.4-desktop.1
```

Punto de partida y adaptaciones

La guía asume un proyecto con carpetas frontend/ y backend/ separadas y una base de datos gestionada con Prisma. El proyecto real difiere en tres aspectos, y cada uno obligó a una adaptación que conviene declarar por delante:

La guía supone	El proyecto tenía	Adaptación
Carpetas frontend/ y backend/	Una sola app Astro con la API en src/server/	Dos Dockerfile en la raíz: Dockerfile.backend y Dockerfile.frontend
Un backend conectado a PostgreSQL	Una API que guardaba tareas en un arreglo en memoria	Repositorio con dos implementaciones: memoria y PostgreSQL
Un frontend que llama a la API	Una lista de tareas solo en estado local	El componente consume la API, con repliegue local si no responde
Migraciones con Prisma	Sin Prisma	CREATE TABLE IF NOT EXISTS al arrancar el backend

Sin estos cambios no habría existido ninguna cadena de punta a punta que contenerizar: el backend no tocaba una base de datos y el frontend no llamaba al backend.

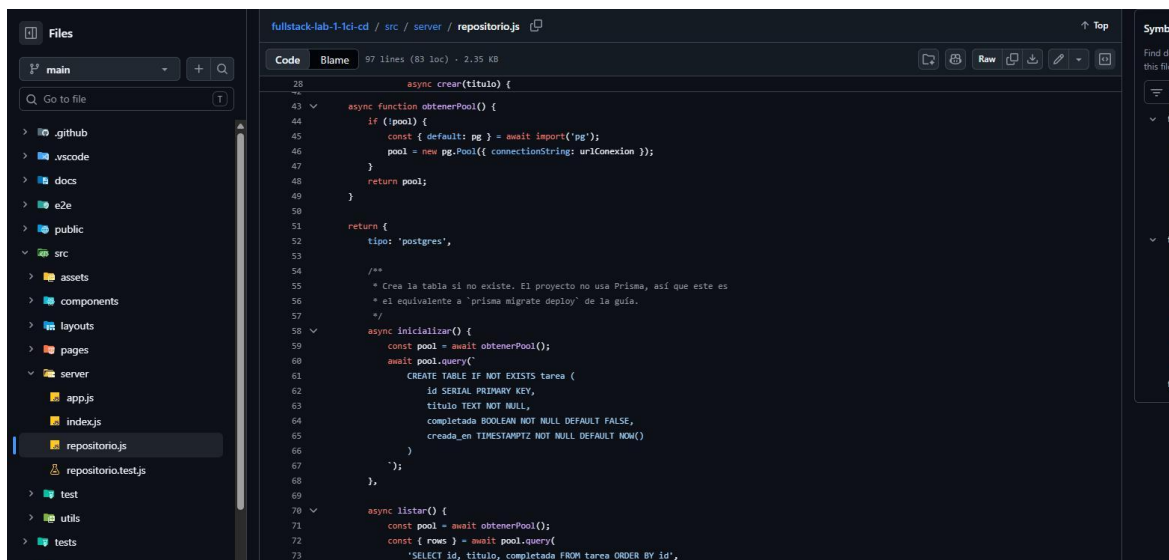
Laboratorio 1 — Dockerfile del backend

Objetivo: escribir el Dockerfile del backend, construir la imagen y verificar que corre de forma aislada antes de integrarla a Compose.

Preparación del backend

La API ya exportaba la app de Express sin llamar a `listen()` (así la usa Supertest desde la Sesión 3). Faltaba un punto de arranque real, y se agregó `src/server/index.js`: crea el repositorio, inicializa la tabla, escucha en el puerto y cierra la conexión de forma ordenada cuando Docker envía SIGTERM al detener el contenedor.

El almacenamiento se movió a `src/server/repositorio.js`, que expone la misma interfaz con dos implementaciones y elige sola: si existe `DATABASE_URL` usa PostgreSQL; si no, un arreglo en memoria. El almacén en memoria es el que usan las pruebas, de modo que la suite sigue corriendo sin base de datos.



```
28      async crear(titulo) {
29
30      }
31
32      async function obtenerPool() {
33        if (!pool) {
34          const { default: pg } = await import('pg');
35          pool = new pg.Pool({ connectionString: urlConexion });
36        }
37        return pool;
38      }
39
40      return {
41        tipo: 'postgres',
42
43        /**
44         * Crea la tabla si no existe. El proyecto no usa Prisma, así que este es
45         * el equivalente a "prisma migrate deploy" de la guía.
46         */
47        async inicializar() {
48          const pool = await obtenerPool();
49          await pool.query(`
50            CREATE TABLE IF NOT EXISTS tarea (
51              id SERIAL PRIMARY KEY,
52              titulo TEXT NOT NULL,
53              completada BOOLEAN NOT NULL DEFAULT FALSE,
54              creada_en TIMESTAMPTZ NOT NULL DEFAULT NOW()
55            );
56          `);
57        },
58
59        async listar() {
60          const pool = await obtenerPool();
61          const { rows } = await pool.query(
62            'SELECT id, titulo, completada FROM tarea ORDER BY id',
63          );
64        },
65      };
66    }
67
68    async function listar() {
69      const pool = await obtenerPool();
70      const { rows } = await pool.query(
71        'SELECT id, titulo, completada FROM tarea ORDER BY id',
72      );
73    }
74  }
75}
```

Figura 1. `src/server/repositorio.js`: la implementación PostgreSQL. El driver `pg` se importa de forma perezosa y la tabla se crea con `CREATE TABLE IF NOT EXISTS`.

.dockerignore

```
node_modules
.env
npm-debug.log
.git
.github
dist
.astro
coverage
playwright-report
test-results
docs
e2e
```

Además de lo que pide la guía (node_modules y .env), se excluyen los artefactos de build, los reportes de pruebas y la documentación: nada de eso debe viajar dentro de una imagen de producción.

Dockerfile.backend

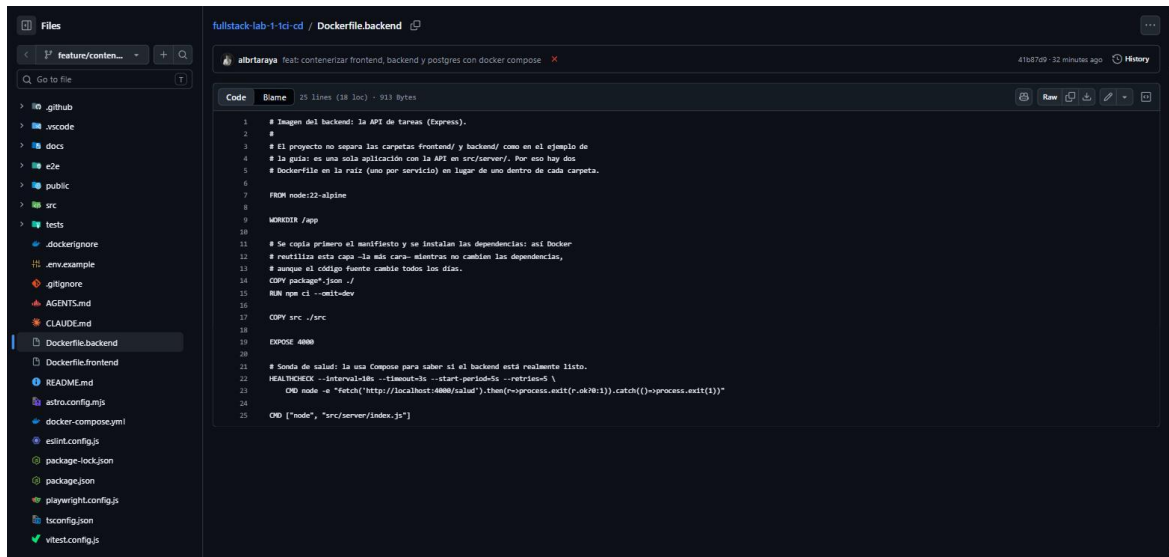


Figura 2. Dockerfile.backend. Primero se copia package*.json y se instalan las dependencias, luego el código: la capa de npm ci queda cacheada entre builds.

Por qué se copia package.json antes que el código: Docker construye por capas y reutiliza las que no cambiaron. Si npm ci depende solo de package*.json, esa capa — la más costosa — se reutiliza mientras no cambien las dependencias, aunque el código cambie todos los días.

Se agregó un HEALTHCHECK que consulta /salud. Compose lo usa para arrancar el frontend recién cuando el backend está realmente listo.

Construcción y ejecución aislada

```
$ docker build -t mi-backend -f Dockerfile.backend .
#10 naming to docker.io/library/mi-backend done

$ docker run -d --name backend-aislado -p 4000:4000 mi-backend

$ docker logs backend-aislado
API de tareas escuchando en el puerto 4000 (almacen: memoria)

$ curl -s http://localhost:4000/salud
{"estado":"ok","almacen":"memoria"}

$ curl -s -X POST http://localhost:4000/tareas -d '{"titulo":"Prueba aislada"}'
{"id":1,"titulo":"Prueba aislada","completada":false}

$ docker ps --filter name=backend-aislado
backend-aislado Up 20 seconds (healthy)
```

Sin DATABASE_URL, el contenedor arranca en modo memoria: justamente lo que se busca en esta prueba aislada, antes de conectar la base de datos en el Laboratorio 3.



```
[{"id": 1, "titulo": "Prueba aislado", "completado": false}]
```

Figura 3. GET <http://localhost:4000/tareas> servido por el contenedor aislado, con la tarea creada en la prueba.

Checkpoint del Laboratorio 1

- ✓ El `.dockerignore` excluye `node_modules` y `.env`.
- ✓ La imagen se construye sin errores con `docker build`.
- ✓ El contenedor responde correctamente al correrlo aislado con `docker run`, y reporta `healthy`.

Laboratorio 2 — Dockerfile multi-etapa del frontend

Objetivo: una etapa construye la aplicación con Node y la segunda sirve únicamente los archivos estáticos con nginx, sin llevar Node a producción.

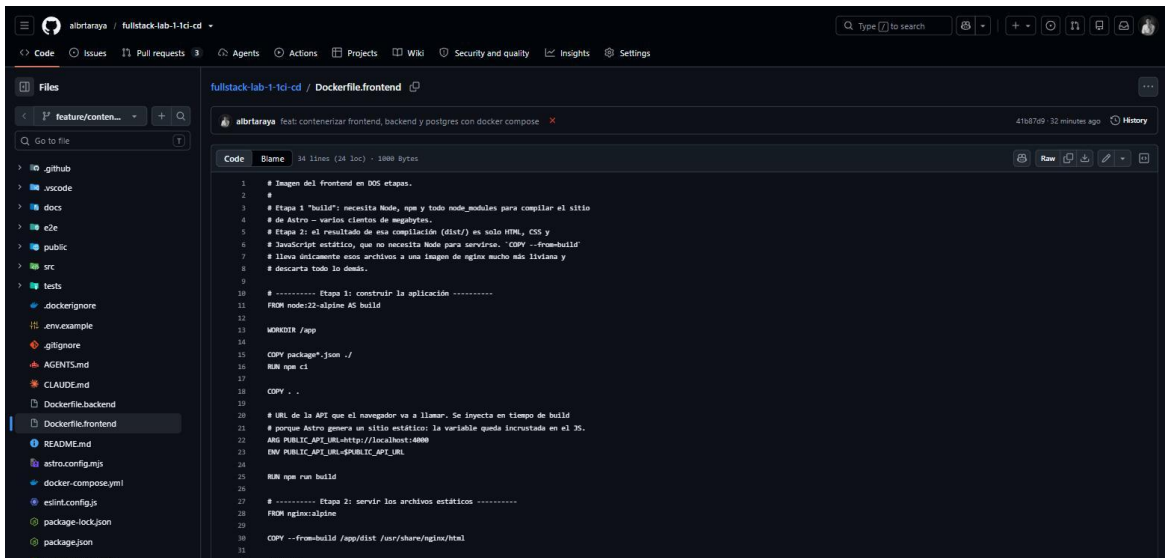


Figura 4. Dockerfile.frontend con sus dos etapas: «build» sobre node:22-alpine y la etapa final sobre nginx:alpine, que solo recibe /app/dist.

La URL de la API se inyecta con ARG PUBLIC_API_URL en tiempo de build: Astro genera un sitio estático, así que el valor queda incrustado en el JavaScript que descarga el navegador.

Ajuste necesario: Node 22

El primer build falló con el ejemplo de la guía:

```
#20 [frontend build 6/6] RUN npm run build
#20 0.773 Node.js v20.20.2 is not supported by Astro!
#20 0.773 Please upgrade Node.js to a supported version: ">=22.12.0"
#20 ERROR: process "/bin/sh -c npm run build" did not complete successfully
```

Astro 7 exige Node 22.12 o superior — el mismo motivo por el que el pipeline usa Node 22 desde la Sesión 2. Ambos Dockerfile pasaron a node:22-alpine.

La prueba del beneficio: tamaño de las imágenes

Para medir qué aporta separar las etapas, se construyó también la etapa «build» por sí sola y se compararon los tamaños:

```
$ docker build -t mi-frontend -f Dockerfile.frontend .
$ docker build --target build -t mi-frontend-etapa-build -f Dockerfile.frontend .

$ docker images
REPOSITORY          SIZE
mi-frontend-etapa-build 572MB
mi-backend          464MB
mi-frontend          69.2MB
```

Imagen	Contenido	Tamaño
Etapla build	Node, npm, node_modules y el código fuente	572 MB
Imagen final	nginx y los archivos estáticos de dist/	69.2 MB

La imagen que llega a producción es 8 veces más pequeña: descarga más rápido, arranca antes y no lleva Node ni dependencias de desarrollo que puedan tener vulnerabilidades.

Ejecución aislada

```
$ docker run -d --name frontend-aislado -p 5173:80 mi-frontend

$ curl -s -o /dev/null -w "HTTP %{http_code}" http://localhost:5173/
HTTP 200

$ curl -sI http://localhost:5173/ | grep -i server
Server: nginx/1.31.5
```

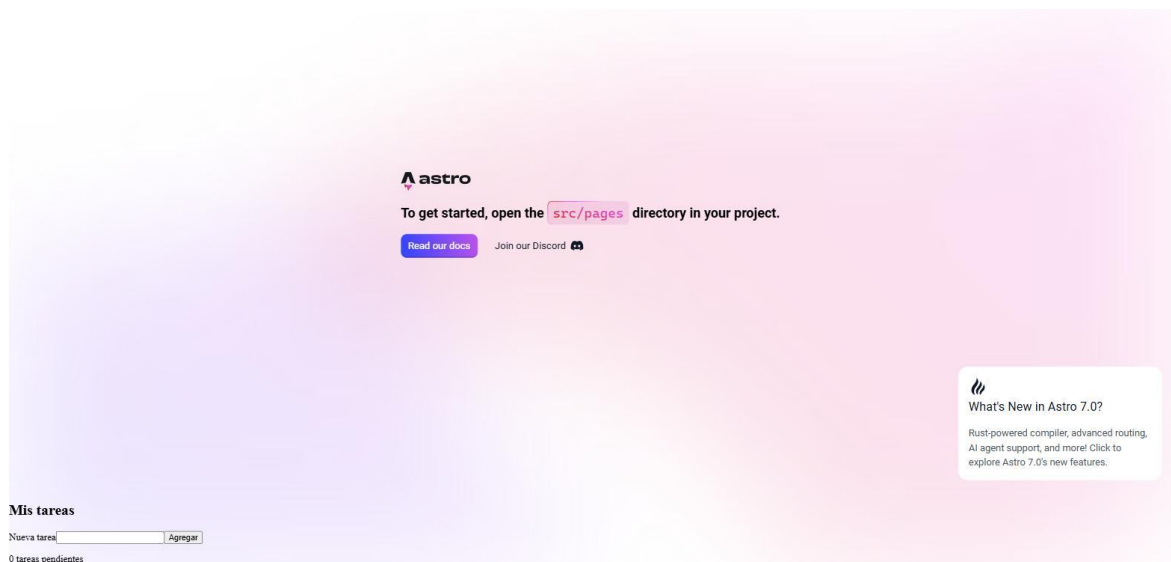


Figura 5. La interfaz servida por nginx en http://localhost:5173. Sin backend muestra «0 tareas pendientes»: la guía anticipa que en esta prueba aislada el frontend todavía no alcanza la API.

Checkpoint del Laboratorio 2

- ✓ El Dockerfile tiene dos etapas claramente separadas (build y nginx).
- ✓ La imagen se construye sin errores, y pesa 69.2 MB frente a 572 MB de la etapa build.
- ✓ El contenedor sirve la interfaz correctamente en el navegador.

Laboratorio 3 — El Puerto de Embarque

Objetivo: orquestar los tres servicios juntos, verificar la aplicación completa contra el PostgreSQL en contenedor, y comprobar la portabilidad real del proyecto.

Eta­pa 1 — Preparar la carga

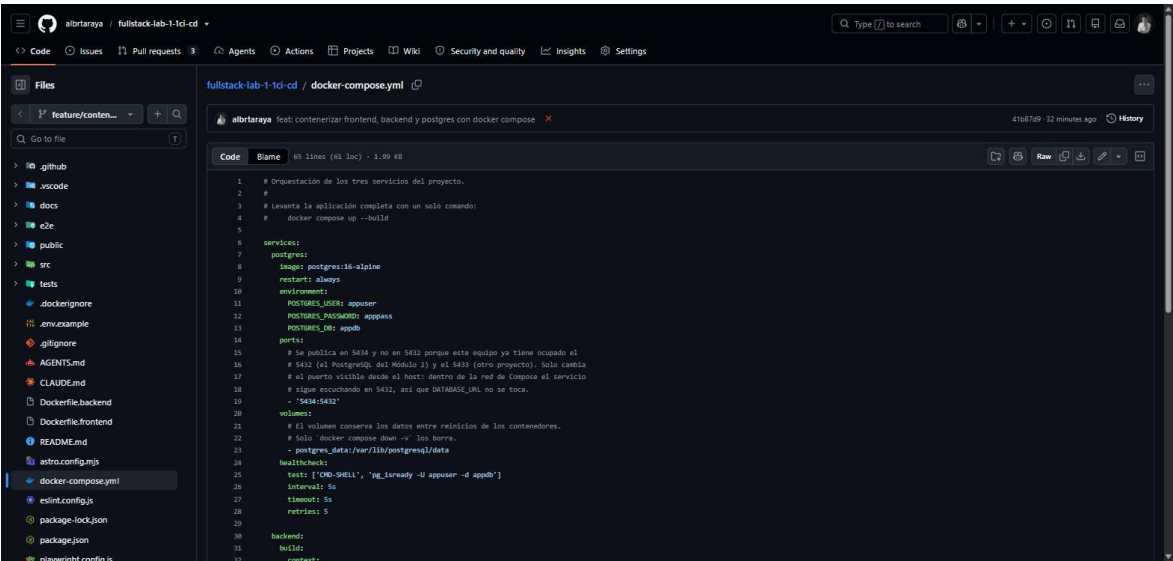


Figura 6. docker-compose.yml (1/2): el servicio postgres con volumen persistente y healthcheck con pg_isready.

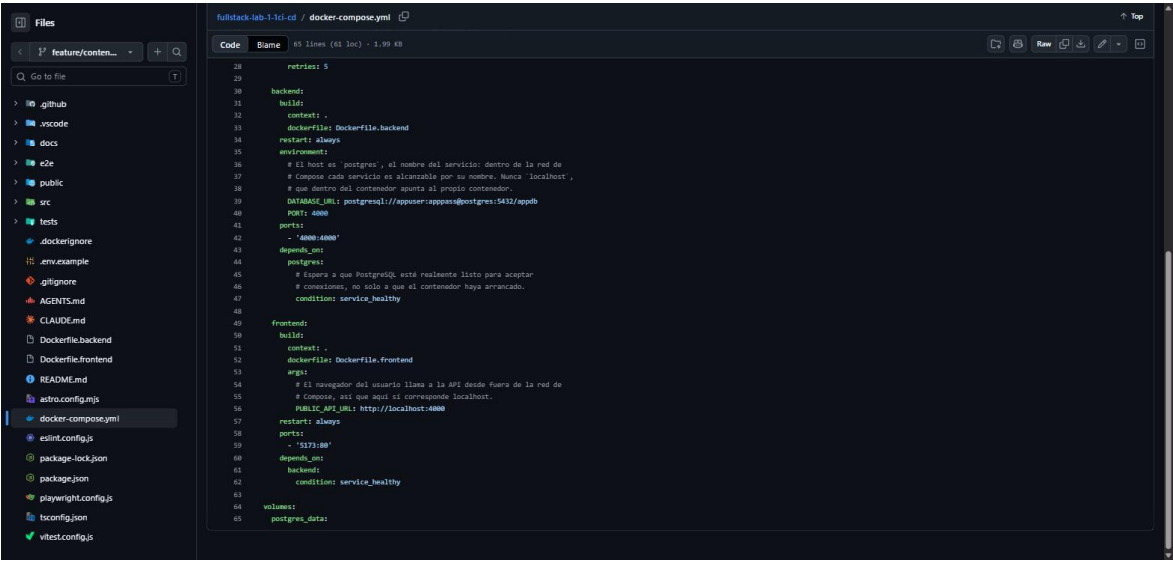


Figura 7. docker-compose.yml (2/2): backend y frontend. DATABASE_URL usa «postgres» como host y cada servicio espera a que el anterior esté healthy.

Puntos clave del archivo:

Decisión	Por qué
DATABASE_URL con host postgres	Dentro de la red de Compose cada servicio se alcanza por su nombre.

	localhost apuntaría al propio contenedor del backend
depends_on con service_healthy	El backend espera a que PostgreSQL acepte conexiones, no solo a que haya arrancado
Volumen postgres_data	Los datos sobreviven a los reinicios de los contenedores
PUBLIC_API_URL con localhost	Aquí sí corresponde localhost: quien llama es el navegador del usuario, fuera de la red

Obstáculo: el puerto 5432 ya estaba ocupado

```
$ docker compose up -d
Error response from daemon: driver failed programming external connectivity
on endpoint lab-2-postgres-1: Bind for 0.0.0.0:5432 failed: port is already allocated

$ docker ps --format "table {{.Names}}\t{{.Image}}\t{{.Ports}}"
fratersys-postgres    postgres:18-alpine    0.0.0.0:5433->5432/tcp
modulo2_postgres      postgres:16-alpine    0.0.0.0:5432->5432/tcp
```

Es el caso previsto en la tabla de problemas frecuentes. El 5432 lo ocupa el PostgreSQL del Módulo 2 y el 5433 otro proyecto; ninguno se detuvo, porque son bases de datos ajenas a este laboratorio. Se publicó el servicio en el 5434. Solo cambia el puerto visible desde la máquina: dentro de la red de Compose PostgreSQL sigue en el 5432, así que DATABASE_URL no se modificó.

Levantar todo

```
$ docker compose up --build -d
Container lab-2-postgres-1 Healthy
Container lab-2-backend-1 Started
Container lab-2-backend-1 Healthy
Container lab-2-frontend-1 Started

$ docker compose ps
NAME                SERVICE    STATUS    PORTS
lab-2-backend-1     backend    Up (healthy)    0.0.0.0:4000->4000/tcp
lab-2-frontend-1    frontend    Up            0.0.0.0:5173->80/tcp
lab-2-postgres-1    postgres    Up (healthy)    0.0.0.0:5434->5432/tcp
```

Migraciones

El proyecto no usa Prisma, así que no aplica `docker compose exec backend npx prisma migrate deploy`. Su equivalente es la inicialización del repositorio: al arrancar, el backend ejecuta `CREATE TABLE IF NOT EXISTS tarea, idempotente`, de modo que no requiere ningún paso manual.

Verificación de punta a punta

```
$ curl -s http://localhost:4000/salud
{"estado":"ok","almacen":"postgres"}
```

El backend confirma que usa PostgreSQL. Después se creó una tarea desde el formulario del navegador. El registro de red del navegador muestra el recorrido:

Petición	Estado	Significado
----------	--------	-------------

GET http://localhost:4000/tareas	200	Carga inicial de la lista
OPTIONS http://localhost:4000/tareas	204	Preflight CORS: el origen 5173 es aceptado
POST http://localhost:4000/tareas	201	La tarea se creó

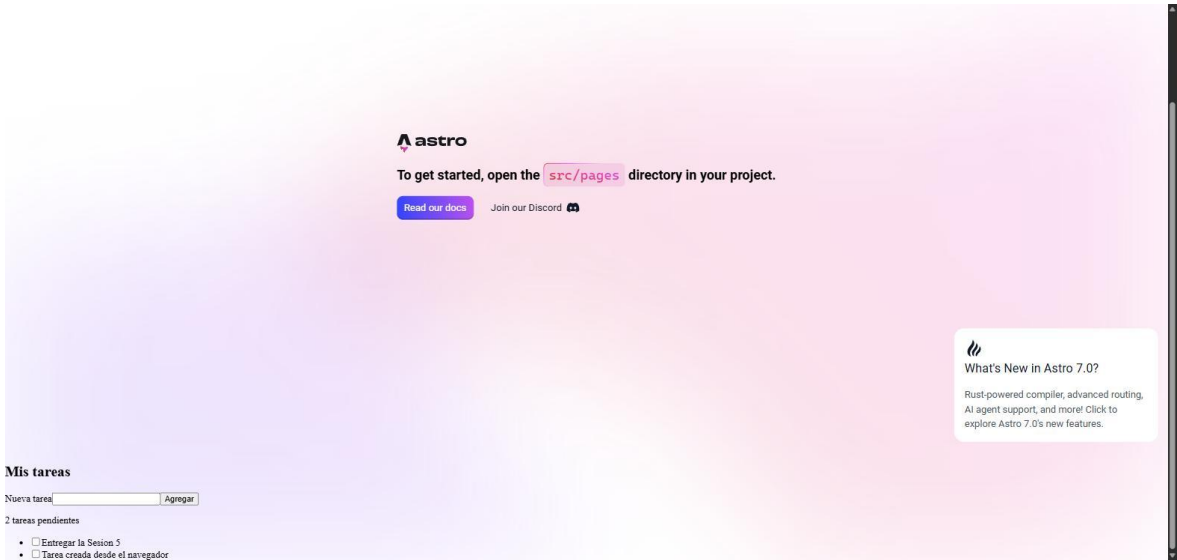


Figura 8. La aplicación completa en `http://localhost:5173` mostrando dos tareas persistidas en PostgreSQL, una de ellas creada desde este formulario.

Y la comprobación directa en la base de datos del contenedor:

```
$ docker compose exec postgres psql -U appuser -d appdb \
  -c "SELECT id, titulo, completada FROM tarea ORDER BY id;"

id |          titulo          | completada
---+-----+-----
 1 | Entregar la Sesión 5      | f
 2 | Tarea creada desde el navegador | f
(2 rows)
```

La fila 2 recorrió todo el sistema: navegador → nginx → Express → PostgreSQL.

Un tropiezo durante esta verificación: el primer intento de crear la tarea no llegó a la base de datos. Era la misma carrera de hidratación diagnosticada en la Sesión 4: el texto se escribió antes de que React tomara el control de la isla de Astro. Tras esperar la hidratación, el POST salió correctamente.

Persistencia del volumen

```
$ docker compose down          # SIN -v
Container lab-2-postgres-1  Removed
Network lab-2_default      Removed

$ docker compose up -d

$ docker compose exec postgres psql -U appuser -d appdb -c "SELECT id, titulo FROM tarea;"
id |          titulo          |
---+-----+-----
```

- 1 | Entregar la Sesión 5
- 2 | Tarea creada desde el navegador

Los contenedores se eliminaron y se volvieron a crear, pero los datos permanecieron: viven en el volumen `postgres_data`, no dentro del contenedor.

Insignia obtenida:  **Maestro de Compose (15 puntos)**

Etapa 2 — Archivo de variables de ejemplo

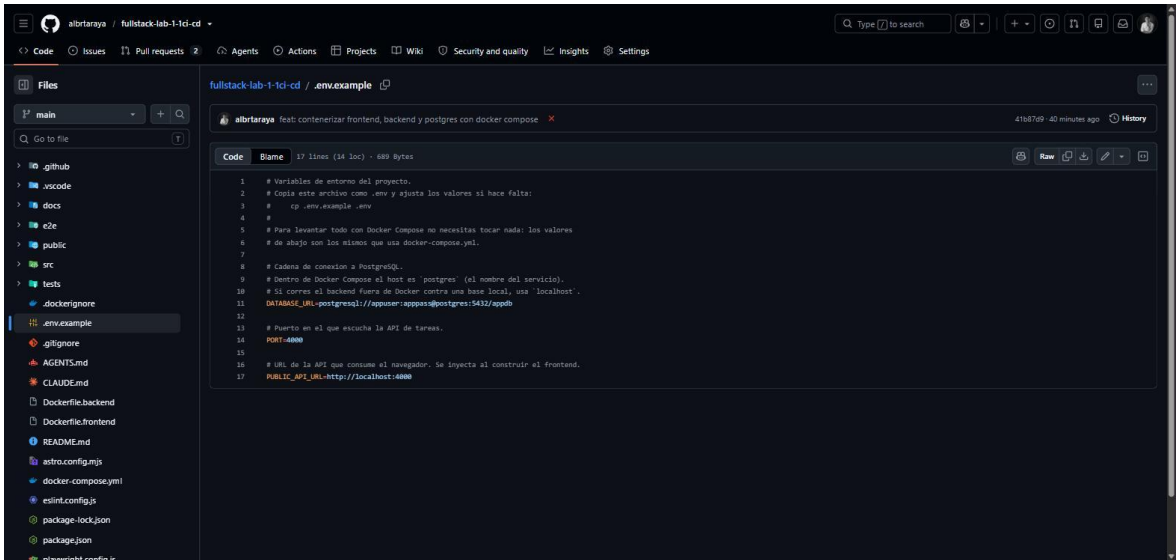


Figura 9. .env.example: documenta cada variable con su propósito, sin credenciales reales. Los valores son los mismos de desarrollo local que usa docker-compose.yml.

```
$ git check-ignore -v .env
.gitignore:5:.env .env
```

El .env real sigue excluido por el .gitignore de la Sesión 1 y no aparece entre los archivos listos para subir.

Integración por Pull Request y quality gate

Como main está protegida, el cambio se propuso en el Pull Request #8. El quality gate de la Sesión 4 lo frenó en el primer intento:

```
Pruebas unitarias y cobertura – Failure
Test Files: 4 passes · 4 total
Test Results: 26 passes · 26 total
Process completed with exit code 1.
```

Las 26 pruebas pasaban, pero el job terminó con error: la cobertura. Los archivos nuevos del backend — el repositorio de PostgreSQL, el endpoint /salud y las ramas de manejo de errores — no tenían ninguna prueba y bajaron el porcentaje global por debajo del umbral. El gate hizo exactamente su trabajo: impedir que entrara código sin probar.

En lugar de relajar el umbral, se escribieron las pruebas que faltaban:

Archivo de prueba	Qué cubre
src/server/repositorio.test.js	Elección del almacén, CRUD en memoria, y consultas a PostgreSQL con el driver simulado — incluido que el título viaje como parámetro \$1 y no concatenado
tests/salud-y-errores.test.js	/salud, respuestas 500 con la base caída, validación previa al acceso a datos, CORS

src/components/ListaTareas.test.jsx	Carga desde la API, alta vía POST y repliegue local ante error o caída de red
-------------------------------------	---

Se excluyó de la medición src/server/index.js: solo lee variables de entorno y llama a listen(); ejecutarlo en la suite abriría un puerto real.

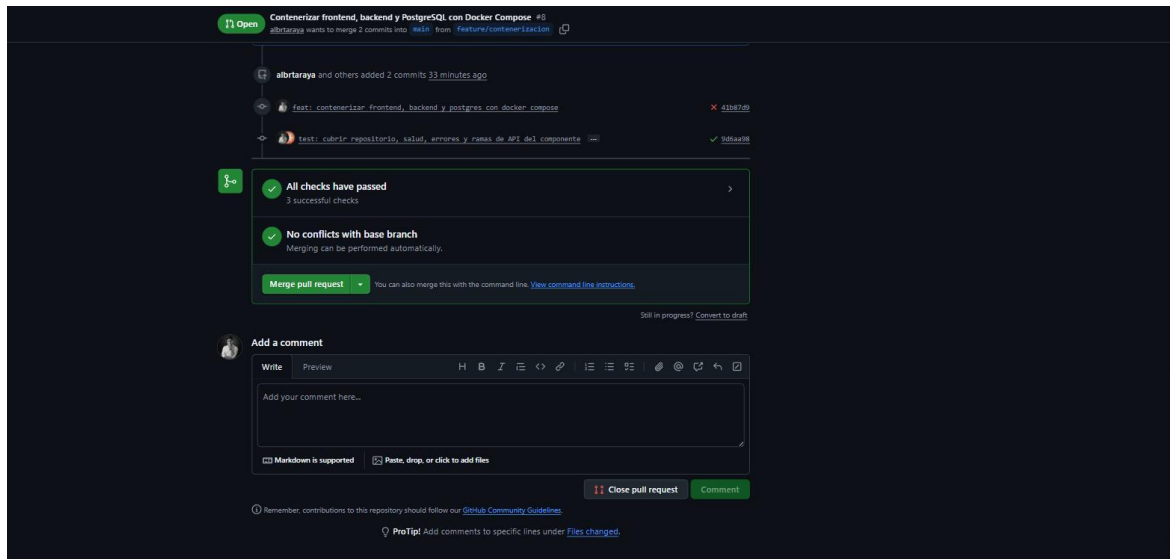


Figura 10. Pull Request #8: el commit inicial marcado en rojo por la cobertura y el segundo en verde. «All checks have passed».

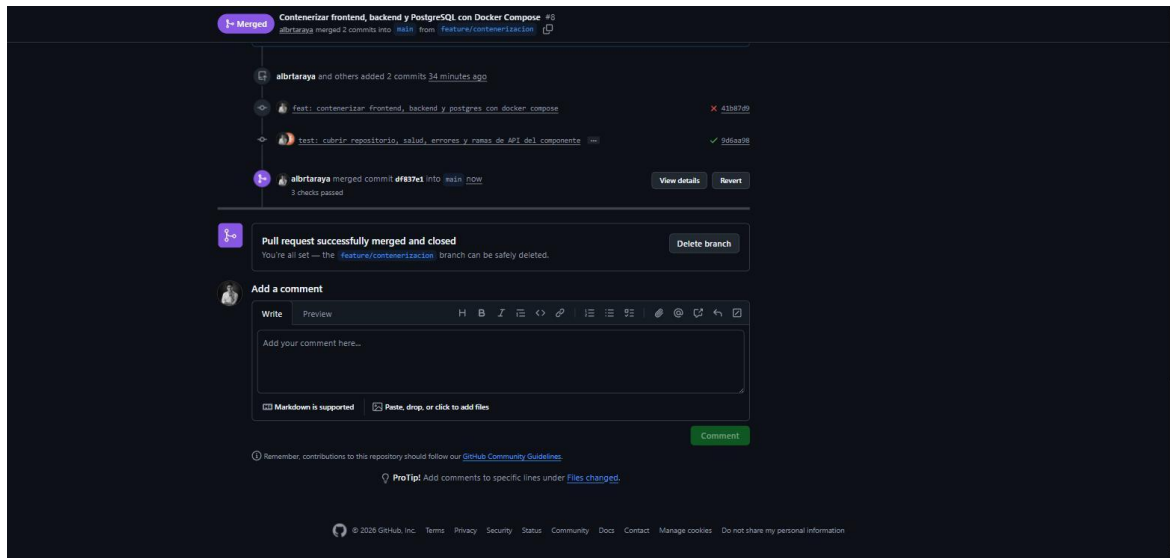


Figura 11. Pull Request #8 integrado a main en el commit df837e1, con 3 checks en verde.

Etapas 3 y 4 — Intercambio de puerto

La actividad se realizó de forma individual, por lo que no hubo un compañero que clonara el proyecto en su propia máquina. Para no dejar sin verificar lo que esa etapa busca comprobar — que el proyecto se levanta sin nada instalado previamente — se reprodujo el procedimiento en una carpeta completamente nueva, siguiendo al pie de la letra los pasos que habría seguido el compañero:

```
$ git clone https://github.com/albrtaraya/fullstack-lab-1-1ci-cd.git
$ cd fullstack-lab-1-1ci-cd

$ ls -a | grep -E "node_modules|^\.env$"
(ni node_modules ni .env: carpeta limpia)

$ cp .env.example .env
$ docker compose up --build -d
Container fullstack-lab-1-1ci-cd-postgres-1  Healthy
Container fullstack-lab-1-1ci-cd-backend-1  Healthy
Container fullstack-lab-1-1ci-cd-frontend-1  Started
```

Ningún npm install ni instalación de PostgreSQL: un solo comando. El clon usa un nombre de proyecto distinto, así que Compose creó un volumen nuevo y vacío — la misma situación que tendría el compañero.

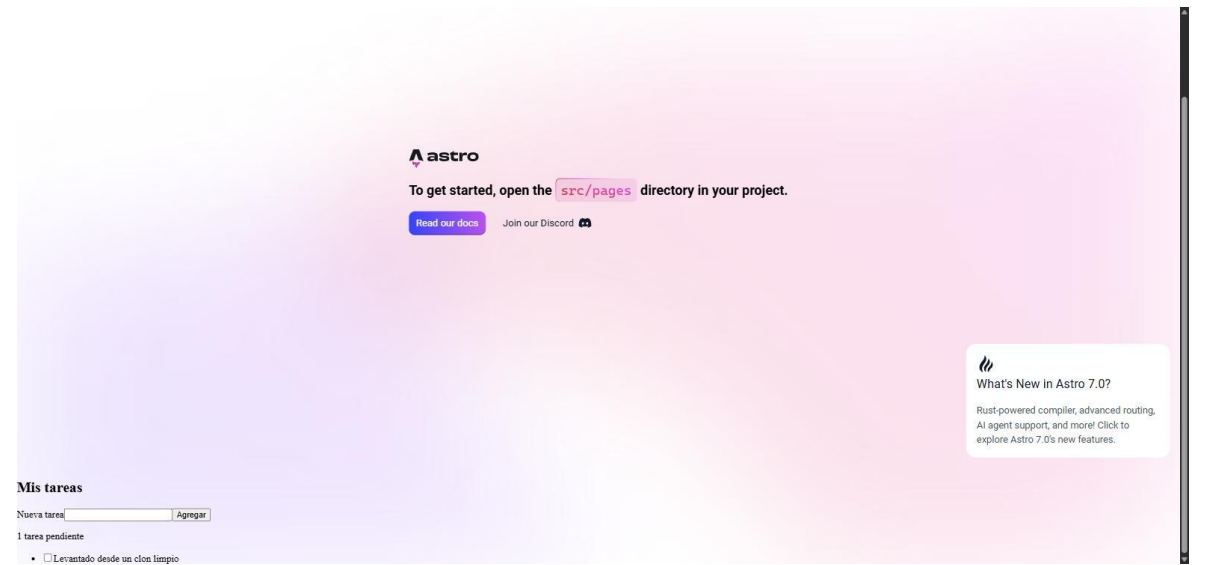


Figura 12. La aplicación levantada desde el clon limpio, con una tarea creada desde el navegador sobre una base de datos recién creada.

```
$ docker compose exec postgres psql -U appuser -d appdb -c "SELECT id, titulo FROM tarea;"
id |          titulo
---+-----
 1 | Levantado desde un clon limpio
```

Verificación del puerto de destino

Ítem	Resultado
El proyecto se levanta sin ayuda adicional más allá del README y .env.example	Cumplido
La aplicación abre en el navegador	Cumplido

Se puede crear una tarea y verla persistir	Cumplido
Errores encontrados	Ninguno en el clon limpio
Validación en la máquina de un compañero	Pendiente

Alcance de esta verificación: demuestra que el repositorio no depende de nada que estuviera en la carpeta de trabajo original. No sustituye la prueba en otra máquina, que es la que además detectaría diferencias de sistema operativo — por ejemplo mayúsculas en rutas. La insignia «Arquitecto del Barco Completo» queda pendiente de esa validación cruzada.

Insignia pendiente:  **Arquitecto del Barco Completo (20 puntos)**

Entregable final de la Sesión 5

Requisito	Estado
Dockerfile del backend funcional, con .dockerignore correcto	Cumplido
Dockerfile multi-etapa del frontend funcional (build + nginx)	Cumplido
docker-compose.yml con los tres servicios conectados correctamente	Cumplido
La aplicación completa funciona de punta a punta con un solo docker compose up	Cumplido
Un .env.example documenta las variables, sin exponer secretos reales	Cumplido
Un compañero confirmó que puede levantar el proyecto en su máquina	Pendiente

Resumen de ajustes respecto a la guía

Ajuste	Motivo
node:22-alpine en vez de node:20-alpine	Astro 7 exige Node >= 22.12.0
PostgreSQL publicado en 5434	5432 y 5433 ocupados por otras bases del equipo
Dos Dockerfile en la raíz	El proyecto no separa frontend/ y backend/
Tabla creada al arrancar en vez de Prisma	El proyecto no usa Prisma
Pruebas nuevas para el backend	El quality gate bloqueó la cobertura

Nota sobre la verificación local de la suite

Durante la sesión la memoria del equipo estuvo al 97 % (0.4 GB libres de 15.9 GB), con la máquina virtual de Docker y otros contenedores activos. En esas condiciones Vitest no lograba iniciar los procesos de las pruebas de componente («Failed to start forks worker»). Las pruebas del backend sí se ejecutaron en local — 22 en verde — y la suite completa se validó en el pipeline, que fue quien la aprobó antes del merge.

Siguiente estación

La Sesión 6 conecta las migraciones al pipeline y mueve las credenciales a GitHub Secrets, cerrando el último punto pendiente antes del despliegue automático de la Sesión 7.

Anexo A — Índice de evidencias

Archivo	Contenido
01-codigo-dockerfile-backend.jpg	Código de Dockerfile.backend
02-backend-aislado-respuesta.jpg	Respuesta del backend corriendo aislado
03-codigo-dockerfile-frontend.jpg	Código de Dockerfile.frontend (dos etapas)
04-frontend-aislado-nginx.jpg	Interfaz servida por nginx de forma aislada
05-codigo-compose-postgres.jpg	docker-compose.yml, servicio postgres
06-codigo-compose-backend-frontend.jpg	docker-compose.yml, backend y frontend
07-codigo-repositorio-postgres.jpg	Código del repositorio PostgreSQL
08-app-punta-a-punta.jpg	Aplicación completa con tareas en PostgreSQL
09-codigo-env-example.jpg	Código de .env.example
10-pr-checks-rojo-a-verde.jpg	PR #8 del rojo por cobertura al verde
11-pr-merged.jpg	PR #8 integrado a main
12-clon-limpio-funcionando.jpg	Aplicación levantada desde un clon limpio

Anexo B — Archivos creados y modificados

CREADOS

.dockerignore	Exclusiones de las imágenes
.env.example	Variables documentadas
Dockerfile.backend	Imagen de la API
Dockerfile.frontend	Imagen de dos etapas
docker-compose.yml	Orquestación de los tres servicios
src/server/index.js	Punto de arranque del backend
src/server/repositorio.js	Almacén en memoria y PostgreSQL
src/server/repositorio.test.js	Pruebas del repositorio
tests/salud-y-errores.test.js	Pruebas de /salud, errores y CORS

MODIFICADOS

src/server/app.js	CORS, /salud y manejo de errores
src/components/ListaTareas.jsx	Consumo de la API con repliegue local
src/components/ListaTareas.test.jsx	Pruebas de las ramas de API
src/test/setup.js	Stub de fetch para pruebas herméticas
vitest.config.js	Exclusión de index.js en la cobertura
package.json	Dependencias pg y cors

Anexo C — Commits

```
41b87d9 feat: contenerizar frontend, backend y postgres con docker compose [cobertura: ROJO]
9d6aa98 test: cubrir repositorio, salud, errores y ramas de API del componente [VERDE]
df837e1 Merge pull request #8 from albrtaraya/feature/contenerizacion
```